

Новый движок Encounter

Go Backend API: обзор, протокол WebSocket
и матрица паритета с ASP.NET-движком

СПЕЦИФИКАЦИЯ	Swagger 2.0 · 478 путей · 450 определений
ИСТОЧНИК	api.en.cx/swagger/doc.json
СНИМОК	31 августа 2026
КЛИЕНТ	пакет <code>encx</code> — один публичный API поверх двух движков

Содержание

1 Обзор: два движка, один клиент

- 1.1 Спецификация
- 1.2 Ключевые отличия от старого движка
- 1.3 Как выбрать движок

2 Протокол WebSocket игрового движка

- 2.1 Подключение
- 2.2 Keepalive и переподключение
- 2.3 Клиент → сервер
- 2.4 Сервер → клиент
- 2.5 Автопереход по таймауту: перевод ленивый
- 2.6 Что покрыто и что нет
- 2.7 Совместимость

3 Матрица паритета движков

- 3.1 Авторизация и сессия
- 3.2 Игровой движок
- 3.3 Каталог и страница игры
- 3.4 Статистика
- 3.5 Команды
- 3.6 Сценарий
- 3.7 Админка: игры и уровни
- 3.8 Админка: содержимое уровня
- 3.9 Админка: управление игрой
- 3.10 Методы, не зависящие от движка
- 3.11 Известные пробелы

1 Обзор: два движка, один клиент

Encounter переезжает с ASP.NET на новый бэкенд («Encounter Go Backend API»). `encx` поддерживает оба движка одновременно: публичный API `encx.Client` не зависит от того, какой движок обслуживает домен.

1.1 Спецификация

- Источник: <https://api.en.cx/swagger/doc.json> (Swagger UI: <https://api.en.cx/swagger/index.html>)
- Копия в репозитории: `docs/newengine/swagger.json`
- Снимок сделан: 2026-08-31
- Версия документа: Swagger 2.0, 478 путей, 450 определений

Обновление снимка:

```
curl -s https://api.en.cx/swagger/doc.json -o docs/newengine/swagger.json
go test ./encx/enapi/ -run TestSpec
```

Тест `TestSpec...` в `encx/enapi` следит за тем, что копия остаётся валидным Swagger-документом и содержит маршруты, на которые опирается клиент.

1.2 Ключевые отличия от старого движка

	Старый движок (ASP.NET)	Новый движок (Go Backend)
Базовый URL	<code>https://{domain}</code>	<code>https://api.en.cx</code> (домен передаётся заголовком)
Контекст сайта	сам хост	заголовок <code>X-En-Domain: {domain}</code>
Авторизация	session-cookie после <code>/login/signin</code>	<code>JWT: Authorization: Bearer <token> + cookie en_access</code>
Формат полей	<code>PascalCase</code>	<code>snake_case</code>
Админка	HTML-формы <code>/Administration/**</code>	REST <code>/admin/games/**</code>
Ошибки	HTML-страница либо <code>Error</code> в JSON	<code>{"error": "...", "message": "...", "code": "...}</code> или <code>{"sentence_key": "...}</code>

Новый бэкенд сохраняет ASP-совместимые маршруты игрового движка (`/gameengines/{controller}/play/{gameId}`), но отвечает моделью `models.EngineState` в `snake_case`, поэтому `encx` выполняет маппинг в привычный `encx.GameModel`.

1.3 Как выбрать движок

По умолчанию — `auto`: клиент один раз спрашивает `GET /sites/domain/{domain}` и запоминает ответ.

Ответ	Вывод
200 с описанием сайта	домен переехал, работаем по новому движку
404 <code>domain_unregistered</code>	домен ещё на ASP.NET

Проверять доступность хоста (`GET /version`) для этого нельзя: один API-хост обслуживает всю зону и отвечает `200` на запрос с любым `X-En-Domain`, включая несуществующий, — такая проба объявила бы переехавшими все сайты сразу.

Две проверки ограничивают, кому клиент вообще готов представиться:

- 1. Хост выводится только для зон Encounter** — `en.cx`, `encounter.cx`, `encounter.ru`, `en-world.org`, `quest.ua`. Иначе отрезание одной метки превратило бы `quest.example.com` в `api.example.com`, то есть отправило бы заголовок сайта, а при входе — логин и пароль на чужой хост. Для домена вне этих зон хост не выводится вообще: в режиме `auto` клиент остаётся на старом движке, а при явном `-engine new` запрос не отправляется и возвращается ошибка с указанием задать `-api-base-url`. Подставлять `api.en.cx` в этом случае нельзя — это легитимный хост Encounter, но он не обслуживает чужой домен, и пароль ушёл бы не туда.
- 2. Сайт должен подтвердить домен** — ответ проверяется по `primary_domain` и списку `domains`. Сайт, который не называет наш домен, отклоняется. Сайт, не назвавший вообще ни одного домена, принимается только если API-хост задал сам вызывающий: этим он за хост поручился.

Кэшируется только определённый ответ. `404` — это «домен ещё на ASP.NET», он не изменится; сетевой сбой или отменённый контекст вердиктом не считаются, чтобы долгоживущий клиент не остался прибит к старому движку из-за одного обрыва.

```
c := encx.New("demo.en.cx") // auto
c := encx.New("demo.en.cx", encx.WithEngine(encx.EngineNew)) // принудительно
```

Либо через окружение — `ENCX_ENGINE=legacy|new|auto` — либо флагом CLI `encli -engine new`. Хост нового движка переопределяется через `encx.WithAPIBaseURL(...)`, `-api-base-url` или `ENCX_API_BASE_URL`.

Матрица паритета методов приведена в разделе «Матрица паритета движков».

2 Протокол WebSocket игрового движка

Адрес: `wss://api.en.cx/games/{gameId}/engine` — тот же путь, что и у REST-режима. Апгрейд выполняется, когда в запросе есть `Upgrade: websocket`; без него ручка работает как REST (`?json=1`).

Реализация: `internal/handlers/engine_ws_handlers.go`, `internal/services/game/engine/hub.go`, модели кадров — `internal/models/engine.go`.

2.1 Подключение

Что	Значение
Авторизация	Authorization: Bearer <jwt>, cookie en_access или ?token=<jwt> — заголовки в браузерном WebSocket недоступны, поэтому query-параметр поддерживается специально
Origin	проверяется как у REST: разрешены домены *.en.cx
Ответ на апгрейд	101 Switching Protocols; при неверном токене — 401 обычным HTTP-ответом
Размер кадра от клиента	не более 4 КБ

Сразу после апгрейда сервер сам присылает `state` с текущим состоянием — отдельный `subscribe` для этого не нужен.

2.2 Keepalive и переподключение

- Сервер шлёт управляющий WebSocket-ping каждые 54 с и ждёт pong; без кадра или pong в течение 60 с соединение закрывается.
- Стандартный клиент отвечает на ping автоматически. Прикладной `{"type": "ping"}` тоже поддержан и отвечает `{"type": "pong"}` — он не ждёт очереди обработки и годится как проверка живости канала.
- Любой входящий кадр продлевает дедлайн чтения.
- Очередь исходящих на клиента — 64 кадра. Клиент, который не успевает читать, отключается: копить состояние живой игры бессмысленно, после переподключения оно всё равно приходит целиком.
- При обрыве: переподключиться с экспоненциальной паузой, получить `state` из первого кадра. Неподтверждённую отправку кода повторять с тем же `req_id` — попытка не спишется дважды (см. ниже).

2.3 Клиент → сервер

Все кадры — JSON-объекты с полем `type`. Поле `req_id` необязательно, но крайне желательно: сервер повторяет его в ответе и использует как ключ идемпотентности.

```
{
  "type": "subscribe", "req_id": "c-17"
} // перезапросить состояние
{
  "type": "answer", "level_id": 812, "level_number": 3, "answer": "код раз", "req_id": "c-18"
}
{
  "type": "bonus", "level_id": 812, "answer": "бонус", "req_id": "c-19"
}
{
  "type": "penalty", "penalty_id": 44, "penalty_act": 1, "req_id": "c-20"
} // 1 - принять, 2 - подтвердить
{
  "type": "ping", "req_id": "c-21"
}
```

Поле	Тип	Примечание
<code>type</code>	string	subscribe answer bonus penalty ping; регистр не важен
<code>req_id</code>	string	эхом возвращается во всех ответах, включая error; ключ идемпотентности мутаций
<code>level_id</code>	int	обязателен для answer и bonus; принимается только текущий уровень игрока, иначе reject_reason: level_changed (в том числе для уже пройденного уровня)
<code>level_number</code>	int	номер уровня (штурм)
<code>answer</code>	string	текст кода
<code>penalty_id</code> , <code>penalty_act</code>	int	подсказка-штраф

Идемпотентность. Повтор мутации с тем же `req_id` в течение **5 минут** возвращает прежний результат с `duplicate: true` и не засчитывает новую попытку. Ключ действует на пару «игра + пользователь» и общий для WebSocket и REST (в REST это поле тела, query `?req_id=` или заголовок `X-En-Request-Id`).

В REST признак повтора приходит **заголовком ответа** `X-En-Duplicate: 1` — тело остаётся обычным состоянием движка, чтобы форма ответа не зависела от того, повтор это или новая обработка. В WebSocket то же самое выражено полем `duplicate: true` в кадре.

Одна задача за раз. Действия одного игрока обрабатываются последовательно: порядок приёма кодов влияет на результат игры. Если предыдущее действие ещё выполняется, новое получает `{ "type": "error", "message": "busy", "req_id": "..." }` — это отказ поставить в очередь, а не отказ игры; корректное поведение клиента — повторить с тем же `req_id`.

2.4 Сервер → клиент

```
{
  "type": "state",      "req_id": "c-18", "event": 0, "state": { /* EngineState */ },
  "type": "event",     "req_id": "c-18", "event": 17, "state": { /* EngineState */ },
  "type": "state",     "req_id": "c-18", "duplicate": true, "state": { /* ... */ },
  "type": "invalidate", "after_ms": 137},
  "type": "timer",     "timer": { "kind": "help", "seconds": 240, "ref_id": 91}},
  "type": "pong",      "req_id": "c-21"},
  "type": "error",     "req_id": "c-18", "message": "bad json"}
```

type	Когда приходит	Что делать клиенту
state	ответ на subscribe, answer, bonus, penalty, а также сразу после подключения	заменить состояние целиком
event	то же, но у состояния ненулевой event (финиш, снятый уровень, автопереход)	заменить состояние и отработать event
invalidate	состояние изменилось не этим клиентом: действие сокомандника, автопереход, случившийся при обращении сокомандника, правка сценария автором	перезапросить состояние через after_ms
timer	вместе с состоянием, если у уровня есть активный отсчёт	завести локальный таймер
pong	ответ на прикладной ping	—
error	кадр не разобран (bad json), неизвестный type (unknown type), очередь занята (busy) или ошибка выполнения	по req_id понять, к какой отправке относится

Поля кадра:

- `req_id` — эхо запроса; у `invalidate` и `timer` отсутствует, они не отвечают на запрос;
- `duplicate: true` — результат взят из журнала обработанных `req_id`: состояние отдано, попытка не списана;
- `after_ms` — сколько подождать перед перезапросом. Значение своё у каждого клиента: событие правки игры приходит всем сразу, и без разброса тысячи клиентов ударили бы в движок в одну миллисекунду;
- `event` — код события (перечень в `EngineState.event`, см. swagger);
- `timer.kind` — `block` (блокировка ответов), `timeout` (автопереход), `help` (подсказка, `ref_id = help_id`).

`invalidate` намеренно не несёт состояния: снимок игры весит килобайты, а изменение может прийти одновременно всей игре. Инициатор действия исключается из рассылки — состояние он уже получил в ответе на свой запрос.

2.5 Автопереход по таймауту: перевод ленивый

Фонового планировщика на сервере нет. Уровень с истёкшим `timeout_expires_unix` переводится не в момент дедлайна, а при следующем обращении к движку — запросе состояния или отправке кода. Пока никто не обратился, игрок остаётся на просроченном уровне, и никакого пуша не будет: переводить некому.

Отсюда обязанность клиента: **дёргнуть состояние сразу после локального `timeout_expires_unix`** (и после возврата приложения из фона, если дедлайн прошёл в это время). Именно этот запрос и выполнит перевод — в ответ придёт `event: 19` (`LevelAutoPassed`), а сокомандники получают `invalidate`. Просроченные подряд уровни дожигаются одним запросом, время каждого перехода записывается дедлайном, а не моментом обращения, поэтому опоздание клиента не искажает статистику игры.

Практическое следствие для редкого страховочного опроса: интервал опроса — это и есть верхняя граница задержки автоперехода. Опрос раз в 2–3 минуты означает, что уровень провисит просроченным до двух-трёх минут, если только клиент не запросит состояние по своему локальному таймеру.

2.6 Что покрыто и что нет

Пушатся: ответы и закрытие секторов сокомандниками, смена уровня, финиш, правка сценария автором, а также автопереход — но только когда его выполнило чужое обращение к движку (см. раздел выше). Не пушатся отдельным сигналом: открытие подсказки по таймеру, появление и истечение бонуса — их клиент отсчитывает сам по абсолютным меткам (`opens_at_unix`, `starts_at_unix`, `expires_at_unix`, `timeout_expires_unix`) из состояния.

2.7 Совместимость

Имена кадров и полей заморожены так же, как REST-контракт: установленные мобильные приложения обновляются месяцами. Добавление поля безопасно, переименование и удаление — только через новый маршрут.

3 Матрица паритета движков

Каждый экспортируемый метод `encx.Client` перечислен ниже: где он живёт в старом ASP.NET-движке и чем обслуживается в новом REST-движке (`docs/newengine/swagger.json`). Пробелы отмечены явно.

Файл сверяется тестом `TestEngineParityMatrixCoversEveryMethod` : метод, который диспетчеризуется по движкам, но отсутствует в таблице, роняет сборку — так документация не может разойтись с кодом.

Спецификация — гипотеза, а не источник истины. `docs/newengine/swagger.json` дважды по ходу этой работы оказалась существенно неверна о развёрнутом API: `prize_cents` описан примечанием, принадлежащим соседнему `price_cents` (на этом построенное умножение на 100 превращало обычный read-modify-write в стократное повышение), а `GET /teams/{id}/members` документирован как массив `models.User` с ключом `id`, тогда как отдаёт строки членства с `user_id` и тремя флагами, которых у `models.User` нет. Оба расхождения найдены замером. Модели в `encx/enapi` с пометкой «Measured against ...» сверены с живым ответом, остальные переписаны из спецификации и могут быть неверны так же — прежде чем строить поведение на непроверенном поле, прочитайте один настоящий ответ.

Утверждения о поведении нового движка в разделе «Известные пробелы» получены живым прогоном на `demo.en.cx`, а не выведены из спецификации. Воспроизвести:

```
ENCX_LIVE_DOMAIN=demo.en.cx ENCX_LIVE_LOGIN=... ENCX_LIVE_PASSWORD=... \
go test ./encx/ -run 'TestLive' -v
```

`encx/live_admin_test.go` создаёт собственную игру, гоняет через неё весь админский CRUD и удаляет её за собой; `encx/live_player_test.go` покрывает игровую сторону. Оба набора пропускаются без учётных данных.

Пути нового движка указаны относительно API-хоста (по умолчанию `api.<зона>`), старого — относительно домена сайта.

3.1 Авторизация и сессия

Метод <code>encx.Client</code>	Старый движок	Новый движок
Login	POST /login/signin?json=1	POST /login (JWT + X-En-Client)
LoginComplete	Login.aspx, затем /login/signin	POST /login + GET /auth/session
VerifyAdminSession	GET /Administration/Games/Level-Manager.aspx	GET /auth/session
GetProfile	GET /UserDetails.aspx (HTML)	GET /auth/session, при необходимости GET /users/{id}

3.2 Игровой движок

Метод <code>encx.Client</code>	Старый движок	Новый движок
GetGameModel	GET или POST-форма на /gameengines/encounter/play/{id}?json=1	тот же маршрут, тело <code>models.EngineClientMessage</code>
GetGameModelLevel	GET /gameengines/encounter/play/{id}?json=1&level=N	тот же маршрут с <code>level=N</code> , ответ — <code>models.EngineState</code> (см. пробел про штурм)
SendCode	POST <code>LevelAction.Answer</code>	POST <code>{"type": "answer"}</code>
SendBonusCode	POST <code>BonusAction.Answer</code>	POST <code>{"type": "bonus"}</code>
GetPenaltyHint	GET <code>?pid=&pact=1</code>	POST <code>{"type": "penalty", "penalty_act": 1}</code>

3.3 Каталог и страница игры

Метод <code>encx.Client</code>	Старый движок	Новый движок
GetGameList	GET /home/?json=1	GET /games/home; для страниц >1 — /games/active и /games/coming
GetDomainGames	/home/?json=1, затем HTML главной	GET /games/home
GetGameDetails	GET /GameDetails.aspx (HTML)	GET /games/{id}/details (JSON вместо HTML)
GetTimeoutToGame	StartCounter со страницы движка	GET /games/{id}/fee-box -> <code>seconds_to_start</code>
EnterGame	GET /MakeGameFee.aspx?confirm=yes	POST /games/{id}/make-fee?confirm=yes
FetchResource	относительные ссылки от домена сайта	относительные ссылки от API-хоста (/media/...)

3.4 Статистика

Метод <code>encx.Client</code>	Старый движок	Новый движок
GetGameStatistics	GET /gamestatistics/full/{id}?json=1	GET /games/{id}/statistics

3.5 Команды

Метод <code>encx.Client</code>	Старый движок	Новый движок
GetTeamDetails	GET /Teams/TeamDetails.aspx (HTML)	GET /teams/{id} (JSON вместо HTML)
GetMyTeamDetails	GET /Teams/TeamDetails.aspx	GET /auth/session -> GET /teams/{id}
GetTeamManagement-Info	разбор ссылок на TeamDetails.aspx	GET /teams/{id}

GetTeamInvitations	разбор своей страницы команды	GET /users/me/team-invitations
AcceptTeamInvitation	?action=accept_invitation	POST /teams/{id}/invitations/respond {accept:true}
RejectTeamInvitation	?action=reject_invitation	POST /teams/{id}/invitations/respond {accept:false}
RequestTeam-Membership	POST /Teams/SendRequest.aspx	GET /teams?q= -> POST /teams/{id}/requests
InviteTeamMember	форма на TeamDetails.aspx	POST /teams/{id}/invitations
RemoveTeamInvitation	?action=remove_invitation	DELETE /teams/{id}/invitations/{userId}
LeaveTeam	ссылка действия на странице команды	POST /teams/{id}/leave
RenameTeam	форма на TeamDetails.aspx	PUT /teams/{id} (read-modify-write)
SetTeamSite	форма на TeamDetails.aspx	PUT /teams/{id} (read-modify-write)
SetTeamForum	форма на TeamDetails.aspx	PUT /teams/{id} (read-modify-write)

3.6 Сценарий

Метод епсх.Client	Старый движок	Новый движок
GetGameScenario	GET /GameScenario.aspx + HTML-парсер	GET /games/{id}/scenario
GetGameScenarioHTML	GET /GameScenario.aspx	нет: HTML-экспорта не существует, метод возвращает ошибку с указанием GetGameScenario

3.7 Админка: игры и уровни

Метод епсх.Client	Старый движок	Новый движок
AdminGetGames	GET /Administration/GamesManager.aspx (Status не заполняется)	GET /admin/games, все страницы
AdminGetLevels	GET /Administration/Games/Level-Manager.aspx	GET /admin/games/{id}/levels
AdminCreateLevels	LevelManager.aspx?levels=create	POST /admin/games/{id}/levels (по одному на уровень)
AdminDeleteLevel	LevelManager.aspx?levels=delete	DELETE /admin/games/{id}/levels/{levelId}
AdminRenameLevels	форма LevelManager	PUT /admin/games/{id}/levels/{levelId}/meta
AdminSwapLevels	LevelManager.aspx?levels=swap&ddlSwapLevels1/2	POST /admin/games/{id}/levels/exchange (результат проверяется — см. пробелы)
AdminInsertLevel	LevelManager.aspx?levels=insert&ddlInsertAfterSrc/Dst	POST /admin/games/{id}/levels/put (результат проверяется — см. пробелы)
AdminCloneLevels	LevelManager.aspx?levels=createlike	POST /admin/games/{id}/levels/copy

AdminGetComment	LevelManager	GET /admin/games/{id}/levels
AdminUpdateComment	форма LevelManager	PUT /admin/games/{id}/levels/{levelId}/meta

3.8 Админка: содержимое уровня

Метод <code>enx.Client</code>	Старый движок	Новый движок
AdminGetTaskIds	LevelEditor.aspx	GET /admin/games/{id}/levels/{levelId}/editor
AdminGetTask	TaskEdit.aspx	тот же editor-документ
AdminCreateTask	POST TaskEdit.aspx	POST .../levels/{levelId}/tasks
AdminUpdateTask	POST TaskEdit.aspx	PUT .../tasks/{taskId}
AdminDeleteTask	ссылка LevelEditor	DELETE .../tasks/{taskId}
AdminGetHintIds	LevelEditor.aspx	editor-документ (helps + penalty_helps)
AdminGetHint	HelpEdit.aspx	тот же editor-документ
AdminCreateHint	POST HelpEdit.aspx	POST .../levels/{levelId}/helps
AdminUpdateHint	POST HelpEdit.aspx	PUT .../helps/{helpId}
AdminDeleteHint	ссылка LevelEditor	DELETE .../helps/{helpId}
AdminGetBonusIds	LevelEditor.aspx	editor-документ
AdminGetBonus	BonusEdit.aspx	editor-документ
AdminCreateBonus	POST BonusEdit.aspx	POST .../levels/{levelId}/bonuses
AdminUpdateBonus	POST BonusEdit.aspx	PUT .../bonuses/{bonusId}
AdminDeleteBonus	ссылка LevelEditor	DELETE .../bonuses/{bonusId}
AdminGetMessageIds	LevelEditor.aspx	editor-документ
AdminGetMessage	MessageEdit.aspx	editor-документ
AdminCreateMessage	POST MessageEdit.aspx	POST .../levels/{levelId}/messages
AdminUpdateMessage	POST MessageEdit.aspx	PUT .../messages/{messageId}
AdminDeleteMessage	ссылка LevelEditor	DELETE .../messages/{messageId}
AdminGetSectorRefs	ALoader/LevelInfo.aspx	editor-документ (sectors)
AdminGetSector - Answers	LevelEditor.aspx?swanswers=1	editor-документ (sectors + answers)
AdminCreateSector	форма LevelEditor	POST .../sectors, затем POST .../answers/batch
AdminUpdateSector	форма LevelEditor	PUT .../sectors/{sectorId} + сверка ответов
AdminDeleteSector	?delsector=	DELETE .../sectors/{sectorId}

AdminAddSector - Answers	форма LevelEditor	POST .../answers/batch
AdminClearLevel - Sectors	перебор ?delsector=	DELETE .../sectors/{sectorId} по каждому сектору
AdminGetLevel - Settings	разбор LevelEditor.aspx	editor-документ
AdminUpdateAutopass	форма LevelEditor	PUT .../levels/{levelId}/autopass
AdminUpdateAnswer - Block	форма LevelEditor	PUT .../levels/{levelId}/settings (section=blocking)
AdminUpdateSector - Completion	форма LevelEditor	PUT .../levels/{levelId}/settings (section=sectors)
AdminGetTeams	выпадающий список forMemberID на TaskEdit.aspx	editor-документ (members)

3.9 Админка: управление игрой

Метод <code>encx.Client</code>	Старый движок	Новый движок
AdminGetGameInfo	GET /Administration/Games/GameEditor.aspx	GET /admin/games/{id}
AdminUpdateGameInfo	POST GameEditor.aspx	PATCH /admin/games/{id} (частичное обновление)
AdminDeliverGame	?action=Deliver	PUT /admin/games/{id}/status — требуется код status_id
AdminNotDeliverGame	?action=NotDeliver	PUT /admin/games/{id}/status — требуется код status_id
AdminAwardPoints	?action=AwardPoints	POST /admin/games/{id}/points/calculate
AdminEndRatings	?action=EndRatings	POST /admin/games/{id}/rate/close
AdminCalculateIK	?action=CalcIK	POST /admin/games/{id}/quality-index
AdminGetCorrections	GET Corrections.aspx	GET /games/{id}/corrections
AdminAddCorrection	POST Corrections.aspx	GET /games/{id}/corrections (резолв имён) -> POST /games/{id}/corrections
AdminDelete - Correction	ссылка Corrections.aspx	DELETE /games/{id}/corrections/{correctId}
AdminGetAction - Monitor	GET ActionMonitor.aspx	GET /games/{id}/monitoring

3.10 Методы, не зависящие от движка

Метод <code>encx.Client</code>	Примечание
<code>AdminCopyGame</code>	оркестрация над другими методами <code>encx.Client</code> — работает на обоих движках без отдельной реализации
<code>AdminWipeGame</code>	оркестрация над другими методами <code>encx.Client</code> — работает на обоих движках без отдельной реализации
<code>ExportCookies</code>	сессия: массив cookies на legacy, объект с <code>apiToken</code> когда задействован новый движок
<code>ImportCookies</code>	принимает оба формата
<code>APIToken</code>	JWT нового движка (пусто на legacy)
<code>SetAPIToken</code>	восстановление ранее полученного JWT
<code>APIBaseURL</code>	хост нового движка
<code>Engine</code>	фактически выбранный движок
<code>EngineMode</code>	запрошенный режим (может быть <code>auto</code>)
<code>SetEngine</code>	смена движка в рантайме
<code>AdminDelay</code>	троттлинг ASP-форм, к REST не применяется
<code>SetAdminDelay</code>	то же
<code>AdminGETDelay</code>	то же
<code>ClearHAR</code>	запись HAR — общая для обоих движков
<code>ClearHARFirst</code>	то же
<code>ExportHARJSON</code>	то же
<code>ExportHARSnapshot</code>	то же
<code>HAREntryCount</code>	то же
<code>SetHARRecordingEnabled</code>	то же
<code>LoginForAntiSpamRecovery</code>	на новом движке сводится к обычному Login: стены <code>NotHumanRequest</code> там нет
<code>LoginViaLoginPage</code>	legacy-only: форма <code>Login.aspx</code> существует только в ASP-движке
<code>ResolveAntiSpamLoginURL</code>	legacy-only: разбор страницы <code>NotHumanRequest.aspx</code>

3.11 Известные проблемы

- `AdminSwapLevels` / `AdminInsertLevel` : на развёрнутом бэкенде маршруты `POST /levels/exchange` и `POST /levels/put` отвечают `204 No Content` и **не меняют порядок уровней**. Проверено на `demo.en.cx` в 13 контролируемых прогонах: одиночная перестановка, повторная, смежных и несмежных уровней, при `levels_sequence_id` 0/1/2, с ожиданием до 5 секунд — порядок не меняется ни в менеджере уровней, ни в редакторе уровня, ни в игровой модели. Передача номеров вместо идентификаторов даёт `404 level not found`, то есть обработчик находит уровни и всё равно ничего не делает. Поэтому обе операции **пересчитывают список уровней и возвращают ошибку**, если порядок не изменился: молчаливый успех оставил бы импорт или редактор уверенными в порядке, которого в игре нет. Перестановка, которая и так уже выполнена, ошибкой не считается, а порядок, ставший отличным и от исходного, и от запрошенного, докладывается отдельно — это уже не «движок проигнорировал», а другой результат. `can_manipulate_levels` из ответа менеджера проверяется до записи всеми пятью методами, меняющими состав уровней (создание, удаление, клонирование, перестановка, вставка), чтобы отказ назывался своей причиной. Замерено, что это безопасно: у начавшихся игр (26365, 26190, 25590, 25487) поле истинно, а ложным для них становится соседнее `can_change_levels_sequence`. Иначе проверка ломала бы правку уровней в идущей игре, которую старый движок разрешает.
- Язык: движок локализует то, что рендерит сам (названия рангов, длительности корректировок), по заголовку `Accept-Language`; параметр `lang` принимают лишь отдельные маршруты (`/games/{id}/statistics`, движковый маршрут). `enapi.Client` шлёт `Accept-Language` из настроенного языка — без него весь API отвечал по-английски там, где старый движок отдавал язык сайта. Паритет при этом условный: `LegacyAdminGetCorrections` жёстко запрашивает `lang=ru` независимо от `WithLang`, поэтому при `WithLang("en")` движки разойдутся — старый всё равно ответит по-русски.
- Постраничные маршруты. Три метода принимают от движка постраничный ответ и не имеют параметра страницы в сигнатуре, поэтому читают все страницы и сообщают об ошибке, если движок так и не дошёл до конца: `AdminGetGames`, `GetGameStatistics` и `AdminGetActionMonitor`. Для статистики это не косметика: `LevelStatInfo.PassedPlayers` и `LevelPlayerCount.Count` вычисляются из самих строк, поэтому неполная страница дала бы не короткий ответ, а неверные итоги.
- `GetGameStatistics` : движок публикует поправки времени отдельным списком `level_corrections`, а не внутри каждой строки, как старый документ. Это агрегат по игре целиком (`GetSummByGameID` в спецификации), повторяющийся на каждой странице, поэтому при склейке страниц он не накапливается — иначе поправка умножалась бы на число прочитанных страниц. Значения сводятся в `StatItem.Corrections` по паре «уровень + участник» со знаком (бонус отрицателен, штраф положителен); агрегатные группы под отрицательными ключами (-1 общее время, -2 чистое) поправок не получают — это уже посчитанные движком итоги. Замер на игре 27053: 486 поправок, все с `user_id` и `level_id`, 483 из них нашли свою строку; остальные принадлежат игрокам без строки на этом уровне.
- `GetGameStatistics` и `needs_confirm` : если автор закрыл статистику, движок отвечает 200 с пустой таблицей и этим флагом. Старый маршрут `full` отдавал таблицу, поэтому запрос повторяется один раз с `confirm=1`. Именно один раз и только в ответ на флаг: сервер записывает такой прос

мотр в журнал, а слать подтверждение на каждое чтение любой игры — побочный эффект, которого у старого вызова не было. Если и после подтверждения флаг стоит, возвращается ошибка, а не пустая таблица.

- Командные операции проверяются после записи, как это делали страницы старого движка: приём и отклонение приглашения, выход из команды, переименование и смена сайта/форума перечитывают состояние и возвращают `*TeamActionError`, если оно не изменилось. Неудача самой проверки (401, таймаут) — не успех, а ошибка «verify state». Членство читается из `GET /teams/{id}/members`, а не из `/auth/session`: сессию можно собрать из предъявленного токена, и тогда она назовёт прежнюю команду — проверка на ней превращала бы каждый удачный приём приглашения и выход в жёсткую ошибку. Сравнение сайта и форума ведётся по нормализованному виду (регистр, схема, хвостовой слэш), потому что серверы ссылки переписывают; «значение просто изменилось» критерием не годится — так прошла бы и подстановка сервером чего-то своего вместо отклонённой ссылки. Добавленные сервером путь и поддомен принимаются, но только с якорем по разделителю: без него `evil-team.example` сошло бы за `team.example`. Форма ответа `GET /teams/{id}/members` взята с живого стенда — там `user_id`, ключа `id` нет; спецификация описывает маршрут как массив `models.User` с ключом `id` и без флагов членства, поэтому читаются оба написания. `InviteTeamMember` и `RemoveTeamInvitation` проверить нечем — списка исходящих приглашений новый движок не публикует (см. пробел про `TeamManagementInfo`), поэтому там успех означает только принятый запрос.
- `Profile.Rank`: новый API публикует только `rank_sentence_key` (справочник `GET /ranks` тоже отдаёт ключи, не строки), поэтому в `Rank` попадает ключ фразы, а не готовая надпись, которую показывала HTML-страница старого движка.
- `AdminCorrection.Reason`: заполняется из `correct_text`, который на корректировках, порождённых бонусами, приходит пустым — колонка «причина» старой страницы для таких строк остаётся пустой.
- `AdminGetCorrections`: `GET /games/{id}/corrections` отвечает 403 для игры, которая ещё не стартовала, тогда как `Corrections.aspx` рисовал пустую таблицу. Замер на `demo.en.cx`: корректировки читаются у любой стартовавшей игры, включая чужие, и отказ приходит только на своей не начавшейся. Пустой список поэтому возвращается лишь при обоих условиях сразу — игра наша И не начата; всё остальное пробрасывается как есть. Признак «игра наша» — `GET /admin/games/{id}` (403 на чужой игре). `GET /admin/games/{id}/lifecycle` для этого не годится: он отвечает 200 на любую игру, в том числе чужую.
- `AdminGameInfo.AuthorComplexity`: соответствует `afc`, умноженному на 10. Измерено на `demo.en.cx`: маршрут принимает `afc` в диапазоне 0..1 включительно (1.0001 отвергается) и хранит значение с шагом 0.1, отбрасывая остаток (0.25 -> 0.2, 0.99 -> 0.9). То есть поле имеет ровно 11 состояний — это в точности целочисленный диапазон 0..10 старой выпадашки `ddlAuthorsComplexity`, чьё умолчание равно 10; спецификация подтверждает соотношение примечанием «SP stores *10». Дробные legacy-значения теряют дробную часть (3.5 -> 3), значение вне 0..10 отклоняется до отправки запроса.
- `AdminGameInfo.Prize`: `prize_cents` названо по колонке, а не по единице измерения — запись `prize_cents=1234` читается обратно как `models.Game.prize = 1234` (проверено вживую). Значение передаётся без пересчёта, как старая форма передавала своё поле `Prize`. Прежнее умножение на 100 превращало обычный read-modify-write в стократное повышение, а за пределом сервера — в

HTTP 400. Осторожно: `GameInfo.Fee / Prize` в каталоге по-прежнему заполняются как `Money{Value: v, Cents: v*100}`; в какой единице сервер хранит саму величину, спецификация не задаёт.

- `AdminLevelSettings` (автопроход): движок хранит награду за таймаут **со знаком** — `timeout_time_award_sec` отрицательное для штрафа и положительное для бонуса (запись `award_is_penalty=true` с 15 минутами читается как `-900`). Старая форма выражала то же парой «галочка + беззнаковая длительность», поэтому знак читается в `TimeoutPenalty`, а модуль — в `Penalty*`.
- `AdminLevelSettings.RequiredSectorsCount`: значимо только при `passing_condition_id == 1`; поле сохраняет прежнее значение при других условиях (0 — все секторы, 2 — очки), поэтому при них возвращается 0, как и на старом движке.
- `AdminGetGames`: админский список постраничный и сообщает `total_pages`. Метод не принимает номера страницы, поэтому читаются все страницы (с потолком в 100 запросов и отбрасыванием повторов) — иначе вызывающий получал бы необозначенный префикс своих игр. Старая страница `GamesManager.aspx` тоже постраничная, но своего пейджера не публиковала, так что усечение там было незаметно и для самого клиента.
- `AdminUpdateSector`: непустой список ответов **заменяет** ответы сектора, пустой — оставляет их нетронутыми (меняется только имя). Так же читается «пустое поле значит не трогать» в остальном API нового движка.
- `ErrSectorStarted`: экспортированный сентинел, на который вызывающие проверяют через `errors.Is`. Новый API не описывает тело ошибки для этого случая, но наблюдаемое условие то же, что проверял старый клиент: удаление не прошло, а сектор остался на уровне. `AdminDeleteSector` перечитывает уровень при ошибке и возвращает `ErrSectorStarted`, если сектор на месте — но только когда движок именно ОТКАЗАЛ (400, 403, 409). 500 или таймаут, которые сектор пережил, — это сбой попытки, а не неудаляемый сектор, и объявлять их сентинелом значило бы увести вызывающего от повтора, который сработал бы. Ответ 404 — «сектора уже нет» — считается достигнутой целью: идентификатор взят из чтения одним запросом раньше, и гонка с чужим удалением не должна ронять очистку уровня (старый клиент перечитывал список каждый круг и просто не видел исчезнувший сектор). `AdminClearLevelSectors` не бросает уровень на первом отказе, а проходит все секторы — но возвращает `ErrSectorStarted`, если хоть один остался: старый цикл с повторами тоже докладывал успех только на пустом уровне, а `cmd/encli/import_scenario.go` пишет новые секторы поверх «очищенного».
- Бонус на нескольких уровнях: `AdminBonus.LevelID` — одно число, поэтому чтение бонуса, привязанного к уровням 3, 5 и 7, вернёт только первый, а обратная запись сузит бонус до него. Старый движок такого бонуса выразить не мог вовсе, но модель нового может, и round-trip через `AdminGetBonus / AdminUpdateBonus` её разрушает.
- `AdminGame.Number`: старый менеджер игр номер не публикует и оставляет 0, новый заполняет его из `game_num`.
- `GetGameList` для страниц >1: первая страница приходит из непостраничного `/games/home`, последующие — из `/games/active` и `/games/coming` с их собственным размером страницы. Если в домашнем блоке окажется не ровно столько игр, сколько в странице каталога, перебор страниц может

пропустить или повторить игру. Унаследовано от исходной реализации, живым прогоном не проверялось.

- `GetDomainGames` : у старого движка была цепочка запасных вариантов (JSON, затем мобильный и десктопный HTML главной), потому что часть доменов публиковала список игр только в разметке. Новый движок отдаёт каталог как данные, поэтому запасных путей нет: домен без игр в каталоге вернёт пустой список.
- `AdminCreateMessage(gameId, levelID, ...)` принимает **идентификатор** уровня, а `AdminUpdateMessage(gameId, levelNum, ...)` — **номер**. Асимметрия унаследована от сигнатур старого движка (там оба значения попадали в один и тот же параметр страницы) и сохранена, чтобы не ломать вызывающих.
- `AdminDeliverGame` / `AdminNotDeliverGame` : новый API меняет статус игры через `PUT /admin/games/{id}/status` с числовым `status_id`, но значения не описаны ни в swagger, ни где-либо ещё, а операция необратима. До получения кодов оба метода на новом движке возвращают явную ошибку и не отправляют запрос; подставить значения можно через `encx.GameStatus-Delivered` и `encx.GameStatusCancelled`.
- `GetGameScenarioHTML` : HTML-экспорта сценария в новом движке нет. Используйте `GetGameScenario`, который работает на обоих.
- `GetGameDetails` и `GetTeamDetails` : на legacy возвращают HTML-страницу, на новом движке — тот же документ в JSON, потому что HTML-эквивалента не существует.
- `GameStatisticsResponse.Level` и `.User` : в `models.GameStatisticsResponse` аналогов нет, поля остаются `nil`.
- `GetGameModelLevel` : параметр `level` выбирает уровень только в играх-штурмах (спецификация: «номер уровня (штурм)», legacy-комментарий говорит то же). В линейной игре оба движка отвечают тем уровнем, на котором игрок сейчас, а запрошенный номер игнорируют — это не расхождение, а смысл параметра. Проверено на `demo.en.cx`: игра 27053 линейная (`levels_sequence_id=0`), и ни одно написание параметра (`level`, `level_number`, `level_id`) уровень не меняет. Поведение в игре-штурме живьём не проверялось: учётная запись не участвует ни в одной из штурмовых игр домена, а вступать в чужую игру ради проверки нельзя.
- `GameStatisticsResponse.PagerVisible` всегда `false` : на старой странице оно значило «таблица показана не целиком», а здесь все страницы уже прочитаны, и предлагать по нему пейджер значило бы звать за строками, которые у вызывающего уже есть.
- `TeamManagementInfo.PendingInvitations` и `.Actions` : описывают HTML-страницу команды; новый движок список исходящих приглашений не публикует, а операции выражает маршрутами.
- `AdminGameInfo` : поля `NotFirstPlaces` и `AcceptRateMode` не имеют соответствия в `models.AdminUpdateGameRequest` и не переносятся. Кроме того, legacy-форма подставляет значения по умолчанию для пустых полей, а REST трактует пустое поле как «не менять».
- `AdminGame.Status` : не заполняется ни на одном движке. Legacy-страница его не публикует, а в админском списке нового движка есть только числовой `status_id`, значения которого не описаны.
- `AdminHint.ReplaceNL` : в `models.AdminHelpRequest` нет соответствующего поля, на новом движке флаг не переносится и читается как `false`.

- `EngineAction.PenaltyAction` и причина отказа (`reject_reason`): в `models.EngineAction-Snapshot` нет данных для `PenaltyActionResult`, поле остаётся `nil`.
- Сценарий: `whole_game_bonuses` (бонусы на всю игру) не попадают в `scenario.Document` — в модели документа нет места для бонусов вне уровня.